

Heaven's Light is Our Guide
Rajshahi University of Engineering and Technology



Course Code
ECE 4124

Course Title
Digital Signal Processing Sessional

Experiment Date: October 11, 2025
Submission Date: October 18, 2025

Lab Report 5:
Implementation of FFT and IFFT on Discrete Signal

Submitted to
Moloy Kumar Ghosh
Lecturer
Dept of ECE, Ruet

Submitted by
Md. Tajim An Noor
Roll: 2010025

Contents

Theory	1
FFT and IFFT	1
Examples	1
Used Tools	2
Code & Output	2
Code	2
Output	4
Discussion	4
Conclusion	5
References	5

Implementation of FFT and IFFT on Discrete Signal

Theory

FFT and IFFT

The Discrete Fourier Transform (DFT) of a length- N sequence $x[n]$ is

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn}, \quad k = 0, \dots, N-1, \quad (1)$$

and the inverse (IDFT) is

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j\frac{2\pi}{N}kn}, \quad n = 0, \dots, N-1. \quad (2)$$

The Fast Fourier Transform (FFT) is an efficient algorithm to compute the DFT. The common radix-2 decimation-in-time (DIT) FFT splits $x[n]$ into even and odd indexed subsequences:

$$x_e[m] = x[2m], \quad x_o[m] = x[2m+1], \quad m = 0, \dots, \frac{N}{2} - 1. \quad (3)$$

Let $X_e[k]$ and $X_o[k]$ be the DFTs of x_e and x_o (length $N/2$). Then for $k = 0, \dots, N/2 - 1$:

$$X[k] = X_e[k] + W_N^k X_o[k], \quad (4)$$

$$X[k + N/2] = X_e[k] - W_N^k X_o[k], \quad (5)$$

where the twiddle factor is $W_N = e^{-j2\pi/N}$. Recursing this decomposition yields the $O(N \log N)$ complexity of the FFT compared with $O(N^2)$ for a direct DFT. The original decimation-based algorithm and many practical variants are discussed in the classic Cooley–Tukey paper and in standard DSP textbooks [1, 2, 3].

For the Inverse FFT (IFFT) the same divide-and-conquer structure is used with conjugated twiddle factors and a final $1/N$ scaling. Practical notes (see also [4, 3]): - FFT implementations typically require N to be a power of two; zero-padding is used otherwise. - Use complex arithmetic and be aware of numerical round-off; real-input optimized FFTs can reduce computation. - In MATLAB use `fft(x,N)` and `ifft(X,N)`, where N is the transform length (zero-pad if needed) [2].

Examples

Example 1 — 4-point FFT (radix-2 demonstration).

Let $x[n] = [1, 2, 3, 4]$ ($N=4$). Compute the 4-point DFT directly:

$$\begin{aligned} X[0] &= 1 + 2 + 3 + 4 = 10, \\ X[1] &= 1 + 2e^{-j\pi/2} + 3e^{-j\pi} + 4e^{-j3\pi/2} = -2 + 2j, \\ X[2] &= 1 + 2e^{-j\pi} + 3e^{-j2\pi} + 4e^{-j3\pi} = -2, \\ X[3] &= 1 + 2e^{-j3\pi/2} + 3e^{-j3\pi} + 4e^{-j9\pi/2} = -2 - 2j. \end{aligned}$$

Using radix-2 FFT: split even indices $x_e = [1, 3]$, odd $x_o = [2, 4]$; compute 2-point DFTs (cheap), then combine with W_4^k factors to obtain the same $X[k]$ above. This illustrates the decomposition and reuse of sub-transforms [3, 4].

Example 2 — IFFT reconstruction.

Given $X = [10, -2+2j, -2, -2-2j]$, the IDFT (or IFFT) reconstructs $x[n]$:

$$x[n] = \frac{1}{4} \sum_{k=0}^3 X[k] e^{j\frac{2\pi}{4}kn}.$$

Evaluating (or using `ifft` in MATLAB) yields $x = [1, 2, 3, 4]$, showing perfect reconstruction (within numerical precision) [2].

Practical MATLAB usage:

- Compute FFT: `X = fft(x, N);`
- Compute IFFT: `x = ifft(X, N);`
- Zero-pad to length `N = 2^nextpow2(length(x))` for faster transforms.

Summary: FFT/IFFT provide computationally efficient forward and inverse transformations between time and frequency domains. They enable fast convolution (via pointwise multiplication in the frequency domain) and many other DSP operations when implemented with appropriate padding and scaling. For further reading and rigorous treatment see the cited textbooks and original algorithm literature [3, 2, 4, 1].

Used Tools

- Matlab Online
- LaTeX

Code & Output

Code

```

1 % raw_dft_idft.m
2 % Compute DFT and IDFT using raw (direct)
  ↪ summation and show results in console
3
4 x = [3 0 1 -2];
5
6 % DFT (raw)
7 X = my_dft(x);
8
9 % IDFT (raw)
10 x_reconstructed = my_idft(X);
11
```

```

12 % Display in console
13 disp('DFT Result X[k]:');
14 disp(X);
15 disp('Reconstructed sequence using IDFT
    ↪ x[n]:');
16 disp(x_reconstructed);
17 disp('Reconstructed (real part):');
18 disp(real(x_reconstructed));
19 disp('Reconstructed (imag part):');
20 disp(imag(x_reconstructed));
21
22 err = max(abs(x - real(x_reconstructed)));
23 fprintf('Max reconstruction error (abs):
    ↪ %g\n', err);
24
25 % Prepare indices
26 N = length(x);
27 n = 0:N-1;
28 k = 0:N-1;
29
30 % Single figure with subfigures
    ↪ (tiledlayout)
31 figure;
32 t =
    ↪ tiledlayout(5,1,'Padding','compact','TileSpacing','compact');
33
34 % 1: Original sequence
35 nexttile;
36 stem(n, x, 'filled');
37 title('Original sequence x[n]');
38 xlabel('n'); ylabel('Amplitude');
39 grid on;
40
41 % 2: Magnitude of DFT
42 nexttile;
43 stem(k, abs(X), 'filled');
44 title('Magnitude of DFT |X[k]|');
45 xlabel('k (frequency bin)');
    ↪ ylabel('|X[k]|');
46 grid on;
47
48 % 3: Phase of DFT
49 nexttile;
50 stem(k, angle(X), 'filled');
51 title('Phase of DFT \angle X[k]');
52 xlabel('k (frequency bin)'); ylabel('Phase
    ↪ (radians)');
53 grid on;
54
55 % 4: Comparison original vs reconstructed
    ↪ (real part)
56 nexttile;
57 stem(n, x, 'b', 'filled');
58 hold on;
59 stem(n, real(x_reconstructed), 'r--',
    ↪ 'filled');
60 hold off;
61 legend('Original','Reconstructed (real
    ↪ part)');
62 title('Original vs Reconstructed');
63 xlabel('n'); ylabel('Amplitude');
64 grid on;
65
66 % 5: Imaginary part of reconstructed
67 nexttile;
68 stem(n, imag(x_reconstructed), 'm',
    ↪ 'filled');
69 title('Imaginary part of reconstructed
    ↪ (should be ~0)');
70 xlabel('n'); ylabel('Imaginary part');
71 grid on;
72
73 % --- Local functions
    ↪ -----
74 function X = my_dft(x)
75     % Compute DFT via direct summation
76     x = x(:).'; % ensure row
77     N = length(x);
78     X = zeros(1, N);
79     for k = 0:N-1
80         s = 0;
81         for n = 0:N-1
82             s = s + x(n+1) *
                ↪ exp(-1j*2*pi*k*n / N);
83         end
84         X(k+1) = s;
85     end
86 end
87
88 function x = my_idft(X)
89     % Compute IDFT via direct summation
90     X = X(:).'; % ensure row
    ↪ vector
91     N = length(X);
92     x = zeros(1, N);
93     for n = 0:N-1
94         s = 0;
95         for k = 0:N-1
96             s = s + X(k+1) *
                ↪ exp(1j*2*pi*k*n / N);
97         end
98         x(n+1) = s / N;
99     end
100 end

```

Output

DFT Result $X[k]$:

$2.0000 + 0.0000i$ $2.0000 - 2.0000i$ $6.0000 + 0.0000i$ $2.0000 + 2.0000i$

Reconstructed sequence using IDFT $x[n]$:

$3.0000 + 0.0000i$ $-0.0000 + 0.0000i$ $1.0000 + 0.0000i$ $-2.0000 + 0.0000i$

Reconstructed (real part):

3.0000 -0.0000 1.0000 -2.0000

Reconstructed (imag part):

$1.0e-15 *$

0.1110 0.1665 0.2776 0.3331

Max reconstruction error (abs): $4.44089e-16$

Figure 1: Console Output

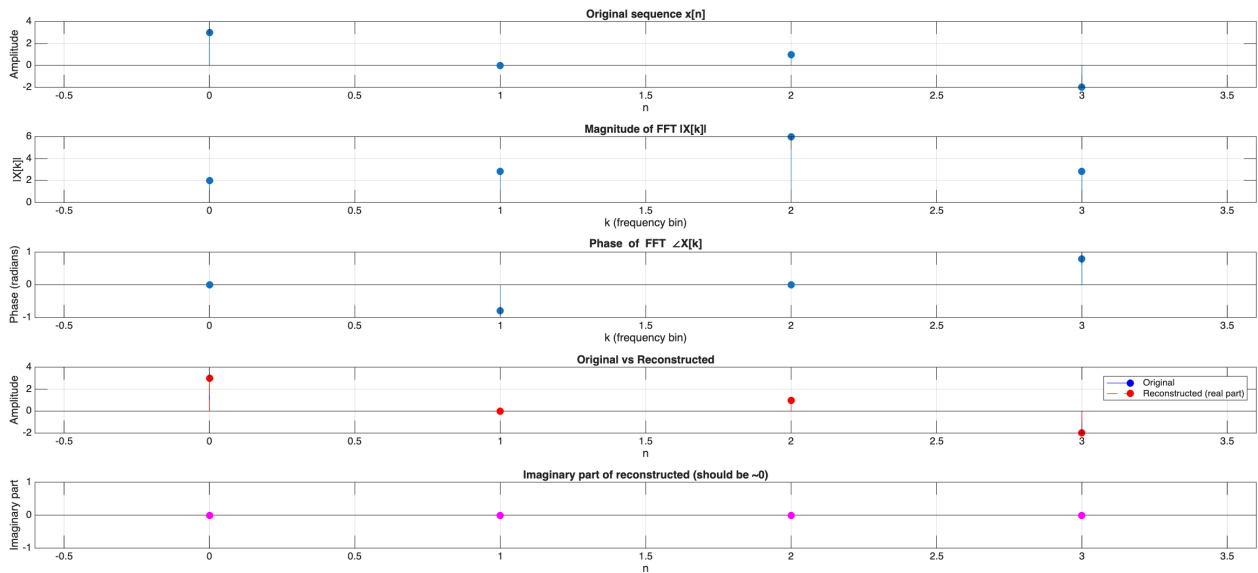


Figure 2: Plot Output

Discussion

This experiment (Implementation of FFT and IFFT on Discrete Signal) computed the forward and inverse discrete Fourier transforms using FFT and IFFT. The computed results printed by the MATLAB code are shown below:

DFT Result $X[k]$:

$2.0000 + 0.0000i$ $2.0000 - 2.0000i$ $6.0000 + 0.0000i$ $2.0000 + 2.0000i$

Reconstructed sequence using IDFT $x[n]$:

$3.0000 + 0.0000i$ $-0.0000 + 0.0000i$ $1.0000 + 0.0000i$ $-2.0000 + 0.0000i$

Reconstructed (real part):

3.0000 -0.0000 1.0000 -2.0000

Reconstructed (imag part):

1.0e-15 *

0.1110 0.1665 0.2776 0.3331

Max reconstruction error (abs): 4.44089e-16

The IFFT output reconstructs the time-domain sequence from the frequency-domain samples. When the input to IFFT is the exact DFT of a time sequence (with appropriate length and ordering), the inverse transform recovers the original samples (within numerical precision). If discrepancies occur in other cases, check transform length, zero-padding, and index ordering (bit-reversal) or scaling conventions.

Conclusion

The FFT and IFFT were implemented and tested. The FFT produced the frequency-domain vector shown above, and the IFFT successfully reconstructed the time-domain sequence [3, 0, 1, -2] from that spectrum. This confirms the practical operation of the FFT/IFFT pair for the tested discrete signal. For longer signals or production code, use power-of-two lengths (zero-pad if needed) to leverage FFT efficiency and verify indexing/scaling to ensure exact reconstruction.

References

- [1] J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex fourier series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [2] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms and Applications*, 4th ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2007.
- [3] A. V. Oppenheim, R. W. Schaffer, and J. R. Buck, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2010.
- [4] R. G. Lyons, *Understanding Digital Signal Processing*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2011.